FIAP • Análise e Desenvolvimento de Sistemas

BioOffset

Plataforma de Compensação de Créditos de Carbono

Documentação técnica detalhada sobre o Pipeline CI/CD, Containerização com Docker, Automação de Testes, Deploy na Nuvem (Azure) e Análise de Desafios Técnicos.

INTEGRANTES DO PROJETO

Gabriel Sabino Lima RM: [563168]

FICHA TÉCNICA DO PROJETO

Linguagem & Framework:	Java 21 • Spring Boot 4 • Spring Data JPA • Spring Security
Banco de Dados:	Oracle Database com migrações via Flyway
Container & Registry:	Docker (Multi-Stage Build Alpine) • Docker Hub
CI/CD & Cloud:	GitHub Actions • Azure Web App for Containers

Descrição do Pipeline CI/CD

Automação completa do ciclo de vida da aplicação: da validação unitária à entrega contínua em nuvem.

FERRAMENTAS UTILIZADAS E PAPÉIS

GitHub Actions:	Orquestrador dos pipelines de CI e CD através de workflows declarativos em YAML.
Docker Hub:	Container Registry responsável por armazenar, versionar e distribuir as imagens públicas/privadas.
Azure Web App:	Plataforma gerenciada (PaaS) que executa o container da aplicação na nuvem com escalabilidade.
GitHub Secrets:	Gerenciamento seguro e criptografado de credenciais do banco (DB_URL, DB_USER, DB_SENHA), token JWT e credenciais de nuvem.

CONTINUOUS INTEGRATION (CI) — CONTINUOUS-INTEGRATION.YAML

Gatilho (Trigger): Disparo automático em qualquer `Pull Request` direcionado para a branch `main`.

Etapas do Workflow:

1. **Git Checkout:** Clona o código-fonte da branch usando a action `actions/checkout@v4`.
2. **Setup Java 21:** Provisiona a SDK do Java 21 distribuição Eclipse Temurin via `actions/setup-java@v4`.
3. **Testes Unitários:** Executa `mvn test` injetando variáveis de ambiente seguras via GitHub Secrets.

Lógica do CI: Atua como um Quality Gate obrigatório. Garante que pull requests com quebras de build ou testes com falha sejam bloqueados antes da integração ao código estável.

CONTINUOUS DEPLOYMENT (CD) — CONTINUOUS-DEPLOYMENT.YAML

Gatilho (Trigger): Disparo automático após qualquer `Push` ou merge aprovado na branch `main`.

Etapas do Workflow:

1. **Job build-and-push:** Realiza checkout, efetua login autenticado no Docker Hub e compila a imagem publicando as tags `latest` e o hash do commit `${{ github.sha }}`.
2. **Job deploy-to-azure:** Dependente do término com sucesso do build (`needs: build-and-push`), atualiza a instância no **Azure Web App for Containers** com o commit SHA gerado.

Lógica do CD: Garante entregas imediatas, automatizadas e idempotentes. O uso da tag SHA assegura rastreabilidade total entre o código no GitHub e o artefato em execução na Azure.

Docker: Arquitetura, Comandos e Imagem

Padronização de ambiente, estratégias de multi-stage build e isolamento da aplicação.

🐳 CONTEÚDO DO DOCKERFILE

```
# Etapa 1: Build da aplicação com Maven e Java 21
FROM maven:4.0.0-rc-5-eclipse-temurin-21 AS build
RUN mkdir /opt/app
COPY . /opt/app
WORKDIR /opt/app
RUN mvn clean package -DskipTests
# Etapa 2: Runtime leve com Alpine JRE
FROM eclipse-temurin:21-jre-alpine
RUN mkdir /opt/app
COPY --from=build /opt/app/target/app.jar /opt/app/app.jar
WORKDIR /opt/app
ENV PROFILE=prd
EXPOSE 8080
ENTRYPOINT ["sh", "-c", "java -Dspring.profiles.active=$PROFILE -jar app.jar"]
```

📐 ARQUITETURA E ESTRATÉGIAS ADOTADAS

Multi-Stage Build: O processo é dividido em etapa de compilação (com Maven e JDK) e etapa de execução (somente JRE). Código-fonte, dependências de compilação e caches do Maven não poluem a imagem final.

Imagem Base Alpine: Uso de eclipse-temurin:21-jre-alpine, reduzindo a imagem de ~800 MB para menos de 190 MB, acelerando downloads no Docker Hub e diminuindo drasticamente a superfície de ataque.

Configuração Dinâmica: O parâmetro -Dspring.profiles.active=$PROFILE permite chavear perfis (dev, prd) em runtime sem reconstruir o container.

💻 COMANDOS DOCKER PRINCIPAIS

Ação / Objetivo	Comando Executado
Build da imagem:	`docker build -t biooffset .`
Execução do container:	`docker run -p 8080:8080 -e URL=... -e USER=... -e SENHA=... biooffset`
Tagueamento:	`docker tag biooffset usuario/biooffset:latest`
Publicação no Registry:	`docker push usuario/biooffset:latest`

Prints do Pipeline Rodando

Espaços reservados para inserção das capturas de tela comprovando a execução dos fluxos automatizados.

EVIDÊNCIA 01 • CI

Build e Testes Unitários no GitHub Actions

[Cole aqui o print do workflow `continuous-integration.yaml` no GitHub Actions, com o step `mvn test` executado com sucesso e todos os testes passando sem erros]

EVIDÊNCIA 02 • CD BUILD & PUSH

Construção e Envio da Imagem ao Docker Hub

[Cole aqui o print da action `docker/build-push-action` concluída com sucesso e a visualização do repositório no Docker Hub exibindo as tags `latest` e `SHA`]

EVIDÊNCIA 03 • CD DEPLOY

Deploy Automatizado no Azure Web App

[Cole aqui o print do job `deploy-to-azure` finalizado com sucesso no GitHub Actions, demonstrando a sincronização com o serviço em nuvem]

Prints dos Ambientes Funcionando

Evidências visuais de disponibilidade da API nos ambientes local/staging e em nuvem na Azure.

EVIDÊNCIA 04 • STAGING / LOCAL

Ambiente Staging / Local Funcionando com Sucesso

[Cole aqui a captura do container Docker em execução local (logs no terminal com inicialização do Spring Boot na porta 8080) e/ou chamada HTTP via Postman, Swagger ou Curl retornando status 200 OK]

EVIDÊNCIA 05 • PRODUÇÃO EM NUVEM

Ambiente de Produção (Azure Web App) Online

[Cole aqui o print do painel do Azure Web App for Containers com status "Running" e a chamada ao endpoint público da aplicação em produção (.azurewebsites.net) respondendo com sucesso]

Desafios Encontrados e Como Foram Resolvidos

Principais obstáculos de arquitetura, segurança e DevOps enfrentados pelo time e soluções implementadas.

🔐 1. GESTÃO SEGURA DE CREDENCIAIS DO BANCO ORACLE

Desafio: A conexão com o banco Oracle corporativo exige credenciais estritas (URL, usuário e senha) que não podem sob hipótese alguma ser versionadas no repositório GitHub por razões de conformidade e segurança.

Solução Adotada: Parametrização via variáveis de ambiente (`${URL}`, `${USER}`, `${SENHA}`) no `application-prd.properties`. No pipeline de CI e no runtime da Azure, os valores são injetados diretamente via **GitHub Secrets** e configurações de ambiente de nuvem, mantendo o código-fonte livre de segredos (princípio de Zero Hardcoded Secrets).

📦 2. TAMANHO EXCESSIVO DA IMAGEM DOCKER E TEMPO DE DEPLOY

Desafio: Uma imagem Docker contendo o JDK 21 completo e o utilitário Maven gerava pacotes superiores a 800 MB, tornando os uploads/downloads no Docker Hub excessivamente lentos e sobrecarregando a inicialização na Azure.

Solução Adotada: Implementação da estratégia de **Multi-Stage Build** com runtime minimalista baseado em `eclipse-temurin:21-jre-alpine`. Apenas o `app.jar` é transferido para o container final, reduzindo o tamanho em mais de 75% e acelerando o tempo de implantação.

🗄️ 3. SINCRONIZAÇÃO DE MIGRAÇÕES FLYWAY NO ORACLE DATABASE

Desafio: Ao instanciar a API em novos containers, o Flyway entrava em conflito com tabelas e objetos previamente existentes no schema Oracle, provocando falha na inicialização do Spring Boot.

Solução Adotada: Adição das configurações `spring.flyway.baselineOnMigrate=true` e `spring.flyway.baseline-version=0` nos properties de produção, permitindo que a aplicação assuma o baseline existente sem travar o deployment.

🏷️ 4. RASTREABILIDADE DE VERSÕES E IDEMPOTÊNCIA DE DEPLOY

Desafio: A utilização exclusiva da tag `latest` dificultava a identificação de qual versão de código estava em execução na nuvem e inviabilizava estratégias rápidas de rollback em caso de incidentes.

Solução Adotada: Configuração do pipeline de CD para taguear simultaneamente a imagem com `latest` e com o SHA do commit (`${{ github.sha }}`). O deploy na Azure aponta diretamente para a tag do SHA, assegurando rastreabilidade total e auditabilidade.

Checklist de Verificação da Entrega

Status de conformidade com os requisitos técnicos e critérios de avaliação do projeto.

Item	OK
Projeto compactado em .ZIP com estrutura organizada Estruturação de pastas limpa e arquivo .zip pronto para submissão final	☑
Dockerfile funcional Multi-stage build configurado (Maven + JDK 21 e runtime leve com Alpine JRE)	☑
docker-compose.yml ou arquivos Kubernetes Orquestração complementar para subida multi-container local	☐
Pipeline com etapas de *build*, teste e *deploy* Workflows do GitHub Actions implementados (continuous-integration e continuous-deployment)	☑
README.md com instruções e prints Documentação de execução Docker, credenciais, pipeline CI/CD e containerização	☑
Documentação técnica com evidências (PDF ou PPT) Relatório técnico estruturado com contextualização, decisões de arquitetura e placeholders	☑
Deploy realizado nos ambientes *staging* e produção Publicação automatizada no Azure Web App for Containers e suporte a ambiente local	☑